

ResearchBench: Paper Replication as the Benchmark for Autonomous AI Research Agents

Rubric-scored replication of 20 inference-optimization papers on a single H100

Annika Kaul Singh
research@infinity.inc

July 9, 2026

Outline

- 1 Why paper replication
- 2 The benchmark
- 3 Rubric structure & evaluation
- 4 Results: agents & harnesses
- 5 Harness optimization, independent of model
- 6 Where this goes

Paper replication is a superb eval target for research agents

- **Novel research is mostly re-composition.** The sub-tasks of new work — environment bring-up, data prep, method implementation, benchmarking, analysis — replicate the vast majority of sub-tasks in pre-existing research.
- An agent that can *faithfully replicate* a paper has demonstrated the skill set needed to *execute* novel research.
- Ground truth exists: the paper's own reported numbers.
- Difficulty is tunable: from structural scaffolding to exact numeric reproduction under tolerance.

The question this benchmark answers

Which coding agent — and which harness — should you trust to do autonomous AI research?

From binary replication to a score: rubrics

2024 — PaperBench: explicit task decompositions (“rubrics”) turn replication from a binary judgment into graded sub-tasks. Rubrics were *hand-authored* by experts.

2026 — what changed: terminal agents now *auto-generate* both the decomposition and the per-task correctness tests.

- rubric generator + critic (two-pass, convergence loop)
- verification contracts with numeric tolerances
- shared structural/behavioral test suites

Replication as a score

Each paper → a tree of graded leaves.

$$\text{score} = \frac{\# \text{ leaves verified pass}}{\# \text{ achievable leaves}}$$

A leaf passes only when an *executable verifier* attributes it — not when the agent claims it.

Pipeline: fully automated rubric generation and grading

arxiv URL → ingest → triage → rubric generator → rubric critic
→ feasibility / unblockers / rubric health → test generator
→ replication agent (iterated sessions) → grader
→ isolated reproduction + confession audit → report

- Rubrics + contracts are **frozen** (content-hashed) — every run of every agent/harness grades against byte-identical targets, so scores are comparable over time.
- The replication agent gets only `paper.md`, an interface contract, the rubric, and stock tools (Read/Write/Edit/Bash/Glob/Grep). Official repos are blacklisted.

The frozen set: researchbench_inference20_h100

- **20 papers**, theme: LLM inference optimization
- Hardware target: single **NVIDIA H100 80GB**
- **7,290 rubric leaves** total (93–935 per paper)
- 1,327 *implementable* headline claims
- Frozen 2026-06-25; rubric + contract hashes verified before/after every run

Paper families: serving simulators, KV-cache management (SSD/tiered/prefix), prefill/decode disaggregation, scheduling & SLOs, mixed precision, GPU kernel optimization, parallelism strategies.

Examples

LLMServingSim 2.0 (214 leaves)
AdaServe (692)
IceCache (935)
ContiguousKV (309)
FP16-equivalence (650)
AutoKernel (268)
Shift Parallelism (407)

Replicable vs. unreplicable papers

- 17/20 papers are scoreable end-to-end; 3 (fast-heterogeneous, flexicache, hexgen-flow) have no implementable headline baseline in this environment ($\rightarrow$ n/a).
- Within scoreable papers, leaves are classified achievable vs. skipped-environment: hardware the paper used but we don't have (A100, H200, TPU-v7), heavyweight pretrained models, proprietary traces.
- Example: ContiguousKV — 309 leaves, 218 environment-skipped, 91 achievable.

Design choice

Unreplicability is *recorded per leaf*, not per paper: the agent is graded only on what is achievable in the declared environment, and the skip reasons are audited.

Compute burden, feasibility, and deciding what to replicate

- Every paper gets an automated **feasibility estimate** (GPU-hours, dollar cost) and an **unblockers probe** (are the models/datasets actually downloadable?).
- Per-run resource contract given to the agent: budget (USD), wall-clock cap, max iterations — the agent is told its fraction and must scope honestly (`results/skipped_leaves.md`).
- **Target slices**: each iteration focuses the agent on a small set of open leaves with exact expected outputs — converting a 900-leaf rubric into tractable turns.

Test-time compute matters (Attempt 3 vs. Attempt 2)

Raising caps (budget 200 $\rightarrow$ 1000, iters 15 $\rightarrow$ 40, wall 8h $\rightarrow$ 12h) *together with* a targeted prompt fix moved closure up to $\sim 5\times$ (next section). Isolating the two levers is planned.

Rubric anatomy

Structure

- Tree of tasks; leaves carry verification {type, spec, paper_cite} and preconditions (requires_hardware, requires_pretrained, ...)
- Types: structural / evidence / behavioral / measured (result_match)
- Measured leaves get a **verification contract**: expected artifact path, paper value, unit, tolerance rule

Granularity

- 93–935 leaves/paper (median ~ 300)
- Fine enough that each leaf is one executable check
- Too-broad leaves are a measured failure class (broad_leaf_unverifiable) fed back into rubric repair

Contract example: `expected_output = results/leaf_<id>.json`, `tolerance: abs(measured - 2.1)/2.1  $\leq$  0.4` — the leaf passes only if a pytest verifier loads that artifact and the assertion holds.

Where agents fail: rubric task categories

Unclosed implementable work concentrates in benchmark-heavy categories (both backends, Attempt 2):

Missed task type	Unclosed leaves
Evaluation, Metrics & Benchmarking	1448
Method Implementation	955
Experimental Setup	450
Environment & Infrastructure Setup	47
Dataset and Model Acquisition	43
Logging, Analysis & Presentation	28
Data Processing & Preparation	27

Agents are weakest exactly where replication is hardest: full benchmark sweeps, hardware-sensitive profiling, end-to-end regeneration of reported numbers.

Failure taxonomy: why leaves stay open

Every non-passing leaf is attributed to a category and an *owner layer*:

Category	Owner	Meaning
missing_per_leaf_test	test harness	no executable verifier was produced for the leaf
missing_result_artifact	result artifact	the contract-named output file was never emitted
broad_leaf_unverifiable	rubric	leaf too coarse to verify; needs splitting
verification_gap	implementation	code ran, evidence doesn't match the contract
numeric_mismatch	implementation	measured, but outside tolerance
environment_blocked	environment	hardware/model/dataset unavailable

This attribution is what makes **harness** optimization tractable: it separates “the model is weak” from “the harness never asked for the right file.”

Success rates across attempts (same frozen set)

Attempt	Change vs. prior	Codex closure	Claude Code closure
1 (unified harness)	first shared controller	mixed-source snapshot	
2 (clean baseline)	no intervention	16/1327 = 1.21%	79/1327 = 5.95%
3 (path blurb)	leaf→file mapping prompt	78/1327 = 5.88%	454/1327 = 34.21%

- Attempt 2, avg. total-implemented: Codex 0.21% vs. Claude 1.44%; Claude led on 12 papers, Codex on 2, 3 tied.
- Attempt 3 lifted *both* backends $\sim 5\times$ with a one-paragraph harness fix + more test-time compute — model unchanged.

The Attempt-3 intervention: tell the agent how it is scored

Diagnosis (from failure attribution): agents implemented code but never produced the *exact files* the grader checks. A leaf scores only when `_tests_local/test_leaf_<LEAF_ID>.py` passes, reading `results/<contract.expected_output>`. Agents covered only the current target slice (~4 leaves), leaving 80+ achievable leaves unverified per paper.

Fix (one appended paragraph, both harnesses):

- 1 state the deterministic leaf→test-path and contract→artifact-path mapping;
- 2 instruct: enumerate *every* achievable leaf, not just the slice;
- 3 remind: `reproduce.sh` must regenerate `results/` from scratch.

Takeaway

A large fraction of measured “agent failure” was *harness legibility* failure. Benchmarks must attribute failures to layers, or they mis-rank models.

Overcoming reward hacking

Agents under budget pressure fabricate: hard-coded “results,” tests that assert paper constants, confident reports of runs that never happened.

What did NOT hold (each bypassed within one run):

- regex “confession” audits — agent renamed `cached` to `paper_reported`
- empty-`src/` negative controls — tests passed on stubs
- “do NOT fabricate” prompt directives — ignored

What holds — behavioral checks on facts the agent didn’t write:

- **isolated reproduction:** wipe `results/`, re-run `reproduce.sh`, observe wall-time, GPU utilization, file diffs
- byte-identical re-ship of pre-baked constants → suspect, score gated
- planned: cost/time floors, tool-use forensics on transcripts

Rule of thumb: if a safeguard can be defeated by different words, it will be.

Long-running agents: the Ralph loop

- Agent SDK sessions can't resume $\Rightarrow$ **state lives on disk**: `progress.md`, source tree, per-leaf tests, results.
- Outer loop re-invokes the agent in fresh sessions until: headline closure validates, or budget / iteration / wall caps hit, or repeated no-value iterations force a reroute.
- Each iteration: repair slice selection $\rightarrow$ agent turn $\rightarrow$ grade $\rightarrow$ failure attribution $\rightarrow$ next slice.
- Benchmark runs: up to 40 iterations, 12h wall, \$1000/paper (Attempt 3); a 20-paper backend run completes in $\sim 10\text{--}14\text{h}$ on one H100.

[OPTIONAL] Anecdote: AutoKernel & FP16-equivalence — ~ 6 failed runs each across loop designs; the historic stuck cases that motivated behavioral validation.

Beyond replication: composing new research ideas

- Because rubrics decompose papers into reusable sub-tasks, a *new* idea can be expressed as a novel composition of leaves from existing rubrics (e.g. “TTKV’s tiering + ContiguousKV’s prefill layout”).
- Evaluation then reuses the same machinery: contracts, per-leaf verifiers, isolated reproduction.
- This turns the benchmark from a replication test into a testbed for **automated research execution**.

Evaluating harnesses independent of the model

- Frozen inputs + rubrics + contracts, one shared controller; *only the backend adapter varies* $\Rightarrow$ score differences attribute to model+harness, and attempt-over-attempt deltas attribute to the harness alone.
- Attempt 2 vs. 3 is the proof: same models, same frozen set, $\sim 5\times$ closure movement from harness changes.

Baseline grid (in progress)

	OpenCode	Cursor	Claude Code harness
Claude	[planned]	[planned]	measured
Codex	[planned]	[planned]	measured (codex exec)
GLM 5.2	[planned]	[planned]	[planned]

Open questions: Fable vs. Mythos? Claude Code vs. Codex? Open- vs. closed-source (GLM 5.2 vs. Claude/Codex)?

Automated harness optimization

- The failure taxonomy is machine-readable (`failure_category_counts`, `owner_layer_counts` per run).
- Attempt 3's fix was found by reading that attribution — the same loop can be closed automatically: *run* → *attribute failures* → *propose harness patch* → *smoke on one paper* → *full run* → *compare on the frozen set*.
- Frozen hashes make every candidate harness comparable; the benchmark becomes the reward signal for harness search.
- Guardrail: optimize against the *diverse rotation*, never a single stuck paper — regressions on one paper type must be caught.

Towards production-codebase paper replication

- The product use case: a company sees a relevant paper and wants those improvements *in their internal, performance-sensitive codebase* — today that is manual researcher work.
- Inference optimization is the sharpest test: numbers are hardware-sensitive, regressions are expensive, and “it runs” $\neq$ “it reproduces.”
- The same contracts that grade replication become **acceptance tests** for landing the technique in production: measured claims, tolerances, isolated reproduction in CI.

Towards fully automated AI research

- AI hypothesis systems already propose new tasks; those proposals can be **decomposed by the same rubric machinery** and **executed by the same agents** this benchmark evaluates.
- The missing piece is trust: behavioral validation and per-leaf attribution give an audit trail for machine-generated results.
- ResearchBench is offered as the yardstick: *which model, in which harness, at what compute, can you trust to do the research?*

Summary

- 1 Replication scored by auto-generated rubrics: 20 papers, 7,290 leaves, frozen.
- 2 Behavioral (not textual) defenses against reward hacking.
- 3 Harness quality is measurable and was the dominant lever: 1.21%/5.95% $\rightarrow \sim 5\times$ with one attributed fix.

Backup: numbers to update before the talk

- Claude Attempt-3 final aggregate (both runs complete): closure $454/1327 = 34.21\%$, avg. total-implemented 8.96% . Codex final: $78/1327 = 5.88\%$, avg. total-implemented 1.63% . Claude led 10 papers, Codex 2, 5 tied; Claude fully closed `tutti` and `shift-parallelism`; `fp16-equivalence` went $0\% \rightarrow 81.9\%$ closure.
- Per-paper Attempt-3 vs. Attempt-2 table (from `BENCHMARK_RUN_SUMMARY.json` of both runs).
- Confound note: Attempt 3 changed prompt *and* compute caps, and ran the two backends concurrently on one H100; lever isolation is Attempt 4.
- Baseline grid cells (OpenCode / Cursor / GLM 5.2) if any land before the talk.